

Final Project: AWS Highly Available 3-Tier Web Architecture

Author: Kylian Labrador

Student ID: 32239363

Affiliation: Dankook University, Dept. of Mobile Systems Engineering

Course: Cloud computing & AWS

Professor: Seehwan Yoo

1. Introduction

As a comprehensive wrap-up for the Cloud Computing & AWS semester, this project focuses on designing, provisioning, and deploying a highly available, scalable, and secure web service using Amazon Web Services (AWS).

The chosen application is a "Minecraft Leaderboard," a practical web service consisting of a decoupled frontend and backend. The primary objective was to move beyond manual console configurations by utilizing **Terraform (Infrastructure as Code)** to deploy an industry-standard **3-tier architecture**. This ensures the infrastructure is reproducible, version-controlled, and resilient.

2. Architecture & Technology Stack

The project was built using the following stack:

- **Frontend (Client Tier):** Angular application hosted statically on **Amazon S3**.
- **Backend (Application Tier):** Python/Flask REST API running on **Amazon EC2** instances.
- **Database (Data Tier):** Relational data managed by **Amazon RDS** (MySQL engine).
- **Infrastructure Provisioning:** HashiCorp **Terraform** (aws provider).

3. Network Design and Security (VPC)

Security and network isolation were prioritized by creating a custom Virtual Private Cloud (VPC) with strict subnets segregation across two Availability Zones (`us-east-1a` and `us-east-1b`) to ensure fault tolerance.

1. **Public Subnets:** Connected to an Internet Gateway (IGW). These subnets host the Application Load Balancer (ALB) and a NAT Gateway.
2. **Private Subnets:** Completely isolated from direct inbound internet traffic. The EC2 instances and the RDS database reside here.
3. **NAT Gateway Configuration:** To allow the backend EC2 instances to download required packages (e.g., Python3, Flask, PyMySQL) during initialization without exposing them to inbound internet requests, a NAT Gateway was implemented in the public subnet and routed to the private subnets.
4. **Security Groups:** Granular firewall rules were established. The ALB accepts HTTP traffic

(port 80) from anywhere (0.0.0.0/0), the EC2 instances only accept traffic (port 5000) originating from the ALB's Security Group, and the RDS instance only accepts MySQL traffic (port 3306) from within the VPC.

4. High Availability and Scalability

To fulfill the core cloud concepts of scalability and high availability, the compute layer was designed to automatically adapt to traffic:

- **Launch Template:** A standardized configuration was created using an Amazon Linux 2023 AMI. Using the `user_data` script, instances automatically install dependencies, pull the Flask application code, and start the web server upon boot.
- **Application Load Balancer (ALB):** Acts as the single point of contact for the frontend, distributing incoming API requests evenly across healthy EC2 targets.
- **Auto Scaling Group (ASG):** The backend is managed by an ASG spanning multiple Availability Zones.

Note regarding the AWS Lab Environment: Due to strict IAM restrictions in the educational AWS Academy/Vocareum environment, dynamic CloudWatch-based scaling policies (Target Tracking) were restricted. To demonstrate scalability, **declarative scaling** was successfully tested via Terraform by modifying the `desired_capacity` variable, which successfully provisioned new instances to the Target Group in under 3 minutes.

5. Benchmarking and Performance Analysis

To validate the infrastructure's resilience under heavy load, a benchmarking test was conducted using Apache Bench (`ab`). The Load Balancer was subjected to a significant traffic spike to observe the network's processing capabilities.

Test Parameters:

- **Total Requests:** 5,000
- **Concurrency Level:** 100 simultaneous connections
- **Target:** ALB Public DNS (`/api/leaderboard/`)

Benchmark Results Summary:

Concurrency Level:	100
Time taken for tests:	6.539 seconds
Complete requests:	5000
Failed requests:	0
Total transferred:	2100000 bytes
Requests per second:	764.67 [# /sec] (mean)
Time per request:	130.776 [ms] (mean)

Analysis: The architecture demonstrated exceptional stability. The Application Load Balancer and the backend EC2 instances successfully handled 100% of the 5,000 requests with **0 network failures**. The system maintained an impressive throughput of **~765 requests per second**, with an average processing time of only **130ms** per request across 100 concurrent users. The data proves that the 3-tier architecture is robust, highly available, and capable of sustaining intensive traffic

without dropping connections.

6. Conclusion

This project successfully synthesized the fundamental concepts of Cloud Computing. By leveraging Terraform to orchestrate VPCs, Load Balancers, Auto Scaling Groups, and managed databases, the resulting architecture is highly secure, scalable, and fault-tolerant. The benchmarking phase empirically validated the design, concluding a successful implementation of a modern AWS-based web service.